HACKERNOON Login Signup Write

Modern API Monitoring for your apps

790 reads

The Complete Guide to AI Agent Memory Files (CLAUDE.md, AGENTS.md, and Beyond)

by **Paolo Perrone** February 27th, 2026

TLDR >

⚡ **Quick Decision Guide**

YOUR SITUATION	WHAT TO USE
Only use Claude Code	`CLAUDE.md` is all you need
Multiple AI tools, solo dev	`AGENTS.md` + `CLAUDE.local.md`
Team with mixed tools	`AGENTS.md` (shared) + `CLAUDE.local.md` (personal)
Enterprise / compliance needs	`Enterprise CLAUDE.md` + `AGENTS.md` + project `CLAUDE.md`

Audio Presented by

catchpoint

HACKERNOON Write

Modern API Monitoring for your apps

One file. Loaded before every conversation. That's all it takes to turn a stateless AI coding assistant into something that actually remembers how your project works.

I figured this out the hard way. Three months into using Claude Code on a production codebase, I was still correcting the same mistakes every session. "No, we use pnpm, not npm." "No, the test command is `make test-integration`, not `pytest`." "No, we don't use default exports here." Every correction vanished when the session ended.

Then I created a CLAUDE.md file. Forty lines of project context. The corrections stopped overnight.

But CLAUDE.md is just one file in what's become an entire ecosystem. There's AGENTS.md, .cursorrules, copilot-instructions.md, CLAUDE.local.md, and now Claude's auto-memory system. Your repo can end up looking like a markdown museum for confused bots.

This guide covers all of them. What each file does, where it goes, and (most importantly) which ones you actually need.

The Problem: Every Tool Wants Its Own File

If you use more than one AI coding assistant (and if you've read my honest tier list of AI agent frameworks, you know why you might), you've probably noticed the mess. Claude wants CLAUDE.md. Cursor wants .cursorrules (or .cursor/rules/). GitHub Copilot wants .github/copilot-instructions.md. Windsurf wants .windsurf/rules. Google's Jules wants JULES.md.

The content is almost identical across all of them. Your coding standards, your build commands, your test setup, your architectural patterns. But you're copying and pasting the same instructions into five different files.

This is the fragmentation problem that AGENTS.md was created to solve. More on that in a minute.

First, let's understand what each file actually does.

HACKERNOON Write

Modern API Monitoring for your apps

team member who has amnesia.

Where it lives (hierarchy matters):

Location	Scope	Shared?
Project root (`./CLAUDE.md`)	This project, all team members	Yes, via git
`.claude/CLAUDE.md`	Same as above, tidier location	Yes, via git
`~/.claude/CLAUDE.md`	All your projects, personal	No
`./CLAUDE.local.md`	This project, just you	No (auto-gitignored)
Enterprise path (`/etc/claude-code/CLAUDE.md`)	All users in org	IT-managed

The hierarchy loads bottom-up. Enterprise policies load first, then your personal preferences, then project-level, then subdirectory-level. More specific instructions override broader ones.

What actually belongs in it:

The /init command generates a starter file based on your project structure. Here's the counterintuitive part: **delete most of what it generates**. The default file includes obvious things (yes, Claude, this is a TypeScript project, I can see that from the package.json). Every line in CLAUDE.md competes for attention with the actual work. (If you've ever wondered why most AI agents fail in production, bad context management is half the answer.)

Target: under 300 lines. Focus on what Claude would get wrong without the file.

A good CLAUDE.md has four sections:

HACKERNOON Write

Modern API Monitoring for your apps

3. **Commands.** Exact strings: `pnpm test:integration` , `make build-docker` , `npm run lint:fix` . Claude uses these verbatim.
4. **Architecture decisions.** "API routes go in /src/api/[resource]/route.ts. We use the repository pattern for database access."

(Everything else? Move it to separate files and use @imports.)

The @imports system:

CLAUDE.md supports importing other files with `@path/to/file` syntax:

```
See @README.md for project overview
See @docs/api-patterns.md for API conventions
See @package.json for available npm scripts
```

Imports can be recursive (referenced files can reference other files, up to 5 levels deep). This solves the "one giant file" problem. Keep your CLAUDE.md lean, move detailed guidance into separate files.

For teams, this is powerful. The frontend team owns `docs/frontend-rules.md` . The security team owns `docs/security.md` . CLAUDE.md just imports them all.

The honest limitation: CLAUDE.md is Claude-only. If your team uses Cursor or Copilot alongside Claude Code, they won't read this file. That's where AGENTS.md comes in.

AGENTS.md: The Universal Standard

AGENTS.md emerged in mid-2025 from a collaboration between Sourcegraph, OpenAI, Google, Cursor, and others. It's now maintained by the Agentic AI Foundation under the Linux Foundation. The pitch is simple: one file, any agent.

HACKERNOON Write

Modern API Monitoring for your apps

schema, no YAML frontmatter required. The closest AGENTS.md to the file being edited takes precedence, and explicit user prompts override everything.

```
# AGENTS.md

## Project Overview
E-commerce platform built with Next.js 14, Postgres, and Stripe.

## Build & Test
- Install: `pnpm install`
- Dev: `pnpm dev`
- Test: `pnpm test`
- Lint: `pnpm lint:fix`

## Code Standards
- Use TypeScript strict mode
- Prefer named exports over default exports
- API routes follow REST conventions in /src/api/

## Testing Requirements
- All PRs must include tests
- Use vitest for unit tests, playwright for e2e
```

AGENTS.md vs CLAUDE.md vs README:

File	For	Purpose
README.md	Humans	Quick start, project description, contributing guide
AGENTS.md	All AI agents	Build steps, test commands, code conventions
CLAUDE.md	Claude specifically	Claude-specific behaviors, preferences, @imports

They're complementary, not competing. README is for humans, AGENTS.md is the universal agent brief, CLAUDE.md adds Claude-specific instructions on top.

HACKERNOON Write

Modern API Monitoring for your apps

The Other Memory Files (Quick Reference)

Not every tool has adopted AGENTS.md yet, and some have features that go beyond what AGENTS.md covers.

.cursorrules / .cursor/rules/*.mdc: Cursor's native format. The newer `.mdc` format supports YAML frontmatter with activation modes (Always, Auto Attached, Agent Requested, Manual). More granular than AGENTS.md, but Cursor-only. Cursor also reads AGENTS.md, so you can use both. Put shared rules in AGENTS.md, Cursor-specific behaviors in .cursor/rules/.

.github/copilot-instructions.md: GitHub Copilot's instruction file. Lives in the .github folder. Copilot also reads AGENTS.md now, so you may not need both.

.windsurfrules: Windsurf's format. Dual structure with global_rules.md for workspace-wide instructions. Windsurf supports AGENTS.md too.

CLAUDE.local.md: Your personal, project-specific preferences that don't get committed to git. Auto-added to .gitignore. Use it for things like your sandbox URLs, preferred test data, or personal workflow quirks. (Your teammates don't need to know you always test with the username "butts123".)

Claude's Auto-Memory: The AI That Takes Its Own Notes

This is the newest addition and the most interesting one. Claude Code now has an auto-memory system where Claude writes notes to itself during your sessions.

It lives in `~/.claude/projects/<project>/memory/` :

```
memory/
├── MEMORY.md          # Index file, loaded every session
├── debugging.md       # Notes on debugging patterns
```

HACKERNOON Write

Modern API Monitoring for your apps

MEMORY.md. You provide the instructions. Claude captures the learnings.

Only the first 200 lines of MEMORY.md load automatically. Topic files (debugging.md, etc.) load on-demand when Claude needs them.

The practical workflow:

When Claude discovers something about your project ("oh, this codebase uses a custom ORM wrapper"), it saves that to auto-memory. Next session, it already knows. No more repeating yourself.

You can trigger this manually too. At the end of a productive session, ask Claude: "Update your memory files with what you learned about our codebase today." The learnings persist.

To review or edit what Claude has saved, run `/memory` during any session.

The honest limitation: Auto-memory is still rolling out. If you don't see it, set `CLAUDE_CODE_DISABLE_AUTO_MEMORY=0` in your environment. And because Claude writes these notes, quality varies. Review them periodically, just like you'd review any documentation.

The /init Then Delete Workflow

This is the fastest way to bootstrap a memory file for any new project.

1. Run `/init` in your project directory
2. Claude generates a starter CLAUDE.md based on your project structure and detected tech stack
3. **Delete what you don't need**

Step 3 is where most people go wrong. The generated file is a starting point, not a finished product. It often includes filler that doesn't add value. ("This project uses JavaScript." Thanks, Claude. I can see the package.json.)

HACKERNOON Write

Modern API Monitoring for your apps

age), don't just correct it once. Tell Claude: "add to my CLAUDE.md: always import from @company/utils-v2, not @company/utils." The instruction persists for future sessions.

Every few weeks, ask Claude to review and optimize your CLAUDE.md. Instructions accumulate, some become redundant, others conflict. A quick cleanup keeps things sharp.

What I Actually Use (My Setup)

After experimenting with all of these on projects ranging from a document Q&A system to an internal agent that kept hallucinating company policy, here's what I settled on:

AGENTS.md in project root with shared instructions (build commands, code standards, testing requirements). This covers any AI tool my team uses.

CLAUDE.md with @imports for Claude-specific behaviors. Stays lean, under 100 lines, mostly pointing to docs/ files.

CLAUDE.local.md for my personal quirks (my preferred test data, sandbox URLs, shortcut commands I use constantly).

Auto-memory enabled. I let Claude take its own notes and review them monthly.

Everything else (.cursorrules, copilot-instructions.md) I've replaced with symlinks to AGENTS.md. One source of truth.

```
# Symlink setup for multi-tool consistency
ln -sfn AGENTS.md .github/copilot-instructions.md
mkdir -p .cursor/rules && ln -sfn ../../AGENTS.md .cursor/rules/main.mdc
```

(Is this elegant? No. Does it prevent instruction drift across tools? Yes.)

Quick Decision Guide

HACKERNOON Write

Modern API Monitoring for your apps

Team with mixed tools	AGENTS.md (shared) + CLAUDE.local.md (personal)
Enterprise/compliance needs	Enterprise CLAUDE.md + AGENTS.md + project CLAUDE.md

The Convergence is Happening

Six months ago, you needed five different files for five different tools. Today, most tools read AGENTS.md. The fragmentation problem isn't solved, but it's getting better.

My bet: AGENTS.md becomes the standard the way README.md did. Not because it's technically superior, but because the Linux Foundation backing and broad tool support create enough gravity to pull the ecosystem together. CLAUDE.md, .cursorrules, and the rest will stick around for tool-specific features, but the shared context will live in one place.

Until then, the symlink hack works fine. And honestly, having a memory file at all puts you ahead of most developers. I wrote an entire guide to building AI agents with LangGraph, and the number one issue people hit wasn't the framework. It was Claude forgetting their project setup between sessions. The gap between "Claude, we use pnpm" every session and "Claude already knows" is the difference between a tool and a teammate.

Proof of Usefulness

Proof of Usefulness Hackathon is a global 6-month developer challenge designed to reward real-world utility projects and initiatives. With 150,000+ in **cash prizes** and **software credits** for winners and $1500+ worth of software and inventory for participants, this is undisputedly the biggest contest of the year. Learn more **here**.

HACKERNOON

Write

Modern API Monitoring for your apps

catchpoin

About Author

Paolo Perrone @paoloap

name@company.com Subscribe

No BS AI/ML Content | ML Engineer with a Plot Twist 🥷 70k+ Followers on LinkedIn

Read my stories

About @paoloap

Comments

Share your thoughts...

More than AI Detector. Preserve what's human.

TOPICS

HACKERNOON

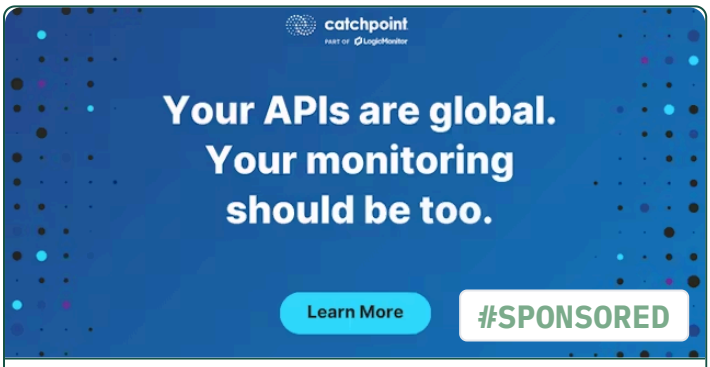
Write

Modern API Monitoring for your apps

THIS ARTICLE WAS FEATURED IN

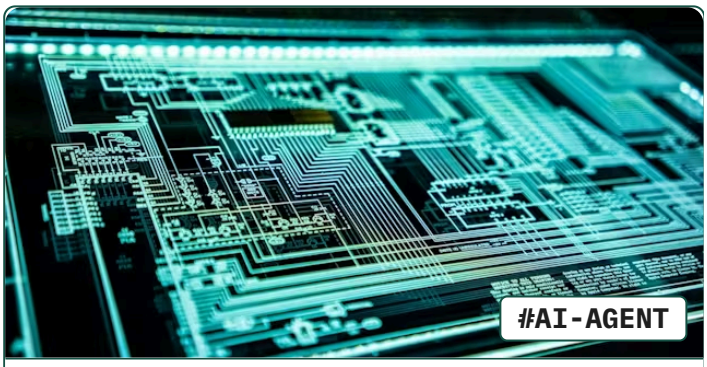
sia · Arweave · ViewBlock · Terminal · Lite · X · Threads · Bsky · Mas

RELATED STORIES



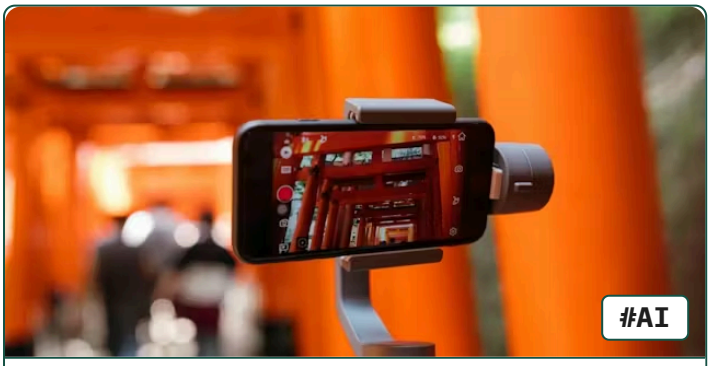
Assure your apps with modern API Monitoring

Catchpoint



5 AI Agent Workflows for Repeatable Success (Code Included)

Paolo Perrone
Aug 20, 2025



How to Trim Video and Audio by Timecode Using fal-ai's Trim Utility

aimodels44
Feb 13, 2026

The Harsh Math of AI: 78% Adoption, 90%+ Disappointment with Generative AI ROI

Michael Jerlis
Sep 26, 2025

HACKERNOON

Write

Modern API Monitoring for your apps

#AI

How AI-Native Engineering Boosts Your App's Scalability

Varsha Ojha
Feb 27, 2026

#AI-AGENT

What is AGENTS.md?

Vladislav Guzey
Aug 22, 2025

THE HACKERNOON NEWSLETTER

Quality Reads About Technology Infiltrating Everything

name@company.com

Subscribe

Yes, I agree to receive electronic content from HackerNoon

Get on **App Store**

Get on **Google Play**

© 2026 HackerNoon. All rights reserved - 56 Edwards Village Blvd, Unit 124-3000-243, Edwards, Colorado 81632, USA

ABOUT

Careers
Contact
Cookies
Emails
HackerNoon AI
HackerNoon CV
HackerNoon Tech
Help
Privacy

READ

Archive
Categories
Editors' Picks
Image Gallery
Leaderboard
Newsletters
On This Day
Signup
Tech Beat

WRITE

Distribution
Editing Protocol
Get Published
Guidelines
Help
New Story
Perks
Post Templates
Prizes

BUSINESS

Blogging Contests
Book Demo Meeting
Business Blogging
Case Studies
Company Directory
Demographics
Hackathons
Live Business Posts

HACKERNOON

Write

Modern API Monitoring for your apps

Say

HACKERNOON